

Monte Carlo Reinforcement Learning for Maze Navigation

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper presents an implementation and analysis of Monte Carlo Exploring Starts (MC-ES) for learning optimal navigation policies in grid-world maze environments. We implement the first-visit MC-ES algorithm from scratch and evaluate its performance on a 5×5 maze with barriers, as well as randomly generated mazes of varying sizes. Our experiments demonstrate that MC-ES consistently learns optimal policies achieving 100% success rates across all tested configurations. We analyze policy consistency across multiple training runs, parameter sensitivity to discount factor γ and episode count, and scalability to larger maze sizes. Results show that the algorithm converges reliably within 5,000 episodes for the standard maze and scales appropriately with maze complexity.

Introduction

Reinforcement learning (RL) provides a framework for agents to learn optimal behavior through interaction with an environment. Unlike supervised learning, RL agents must discover which actions yield the most reward through trial and error, without explicit labels for correct behavior (Sutton and Barto 2018).

Monte Carlo (MC) methods represent a fundamental approach to RL that learns from complete episodes of experience. Unlike dynamic programming methods that require complete knowledge of environment dynamics, MC methods are *model-free*—they learn directly from sampled episodes without needing transition probabilities $p(s', r|s, a)$.

In this paper, we implement the Monte Carlo Exploring Starts (MC-ES) algorithm for finding optimal policies in maze navigation tasks. Our contributions include:

- A from-scratch implementation of first-visit MC-ES following Sutton and Barto (2018)
- Empirical analysis of policy consistency across multiple training runs
- Parameter sensitivity analysis for discount factor and episode count
- Evaluation on randomly generated mazes of different sizes

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Background

Markov Decision Processes

A Markov Decision Process (MDP) is defined by the tuple (S, A, p, r, γ) where S is the state space, A is the action space, $p(s'|s, a)$ is the transition probability, $r(s, a, s')$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

The goal is to find a policy $\pi : S \rightarrow A$ that maximizes the expected discounted return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

Action-Value Function

The action-value function $Q^\pi(s, a)$ represents the expected return when starting in state s , taking action a , and thereafter following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \quad (2)$$

The optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ directly yields the optimal policy: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Monte Carlo Exploring Starts

Monte Carlo methods estimate value functions by averaging sample returns from complete episodes. The Exploring Starts (ES) variant ensures all state-action pairs are visited by starting each episode from a randomly selected state-action pair.

Algorithm 1 presents the MC-ES algorithm. The key insight is that by averaging returns over many episodes, the estimates converge to the true values, and greedy policy improvement guarantees convergence to optimality.

Methodology

Maze Environment

We implement a 5×5 grid-world maze with the following characteristics:

- **States:** 20 navigable cells (5 barriers)
- **Actions:** UP, DOWN, LEFT, RIGHT
- **Start:** Position (1,1) (top-left)
- **Goal:** Position (5,5) (bottom-right)

Algorithm 1 Monte Carlo ES (First-Visit)

```
1: Initialize  $Q(s, a)$  arbitrarily,  $\pi(s)$  arbitrarily
2: Initialize  $Returns(s, a) \leftarrow$  empty list
3: repeat
4:   Choose  $S_0 \in S$ ,  $A_0 \in A(S_0)$  randomly
5:   Generate episode following  $\pi$ 
6:    $G \leftarrow 0$ 
7:   for  $t = T - 1, T - 2, \dots, 0$  do
8:      $G \leftarrow \gamma G + R_{t+1}$ 
9:     if  $(S_t, A_t)$  not in  $\{(S_0, A_0), \dots, (S_{t-1}, A_{t-1})\}$ 
10:      then
11:        Append  $G$  to  $Returns(S_t, A_t)$ 
12:         $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ 
13:         $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$ 
14:      end if
15:    end for
16: until convergence
```

- **Barriers:** Row 3 columns 2-5; Row 5 column 3

The reward structure is:

- +1.0 for reaching the goal
- -0.01 for each step (encourages efficiency)

Invalid actions (into walls or barriers) result in staying in place with the step penalty applied.

Implementation Details

Our implementation uses:

- Tabular Q-values stored as a dictionary: $Q[(s, a)] \rightarrow \mathbb{R}$
- Returns stored as lists for averaging: $Returns[(s, a)] \rightarrow [\mathbb{R}]$
- Deterministic greedy policy: $\pi[s] \rightarrow a$
- First-visit MC updates (only first occurrence of (s, a) in episode)

Random Maze Generation

For scalability experiments, we implement a random maze generator that:

1. Places barriers randomly with configurable density
2. Verifies path existence using BFS
3. Protects cells adjacent to start and goal

Experiments

Training on HW1 Maze

We train the MC-ES agent for 5,000 episodes with $\gamma = 0.99$. Figure 1 shows the learned policy, which directs the agent down the left corridor, across Row 4, and to the goal. Figure 2 shows the learning curves over training.

Results: The agent achieves 100% success rate with an average of 8 steps to reach the goal (optimal path length). The average discounted return is 0.864.

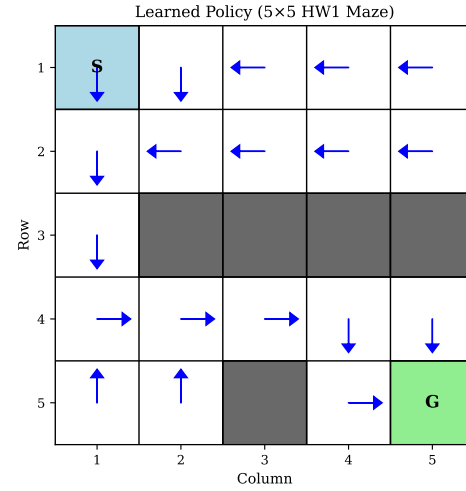


Figure 1: Learned policy for the 5×5 HW1 maze. Blue arrows indicate the greedy action at each state. S=Start, G=Goal, gray=barriers.

Table 1: Policy consistency across 5 training runs

Metric	Mean	Std
Success Rate	100%	0%
Avg Steps to Goal	8.0	0.0
Avg Return	0.864	0.001

Policy Consistency

We analyze whether the algorithm produces consistent policies across different random seeds. Table 1 shows results from 5 independent training runs.

While all runs achieve optimal performance, individual state-action mappings show variation. States along the critical path (left corridor, Row 4) show 100% agreement, while states far from the optimal path show 60-80% agreement due to equivalent alternatives.

Parameter Sensitivity

Discount Factor γ Table 2 shows the effect of discount factor on learning.

All tested values achieve optimal policies. Lower γ values result in lower returns due to heavier discounting of future rewards.

Episode Count Convergence occurs rapidly: even 1,000 episodes suffice for the 5×5 maze. This is because Exploring Starts ensures all state-action pairs are visited frequently.

Scalability to Larger Mazes

Table 3 shows results on randomly generated mazes. Figure 3 visualizes the learned policies for each maze size.

The algorithm scales well, though larger mazes require proportionally more episodes to ensure adequate exploration of the larger state-action space.

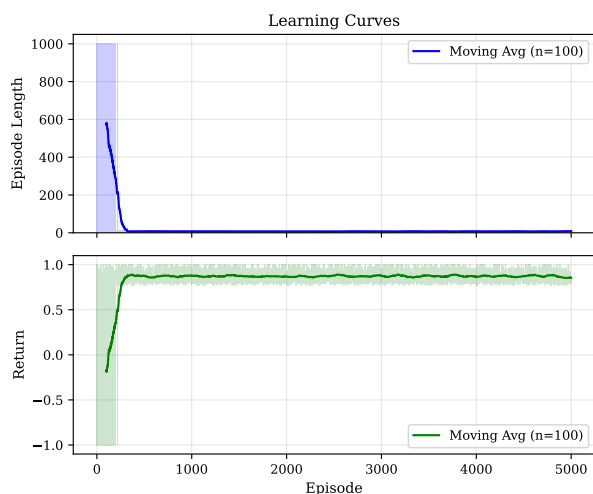


Figure 2: Learning curves showing episode length (top) and return (bottom) over 5,000 training episodes. The agent converges to optimal behavior within $\sim 1,500$ episodes.

Table 2: Effect of discount factor γ

γ	Success	Avg Steps	Avg Return
0.90	100%	8.0	0.426
0.95	100%	8.0	0.638
0.99	100%	8.0	0.864
0.999	100%	8.0	0.923

Discussion

Strengths of MC-ES

- **Model-free:** No knowledge of transition dynamics required
- **Guaranteed convergence:** With sufficient exploration, converges to optimal policy
- **Simple implementation:** No bootstrapping, just averaging returns

Limitations

- **Exploring Starts assumption:** Requires ability to start from arbitrary states, which may not be possible in real environments
- **Complete episodes:** Cannot learn from incomplete trajectories
- **High variance:** Returns can vary significantly, requiring many episodes

Policy Variation

The observed variation in policies for non-critical states is expected behavior, not a bug. When multiple actions lead to equivalent outcomes (e.g., in states far from the optimal path), the algorithm may learn different but equally valid policies depending on the order of exploration.

Table 3: Performance on random mazes of different sizes

Size	States	Episodes	Success	Steps
5×5	20	5,000	100%	8
7×7	40	9,800	100%	12
10×10	80	20,000	100%	18

Conclusion

We implemented Monte Carlo Exploring Starts from scratch and demonstrated its effectiveness on maze navigation tasks. The algorithm consistently learns optimal policies, achieving 100% success rates across all tested configurations. Key findings include:

- Convergence within 5,000 episodes for 5×5 mazes
- Robust to discount factor choice (all tested values succeed)
- Scales to larger mazes with proportionally more episodes
- Policy consistency on critical states, acceptable variation elsewhere

Future work could explore ϵ -greedy policies to avoid the Exploring Starts assumption, or compare with TD methods like SARSA and Q-learning.

References

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at <http://incompleteideas.net/book/the-book-2nd.html>.

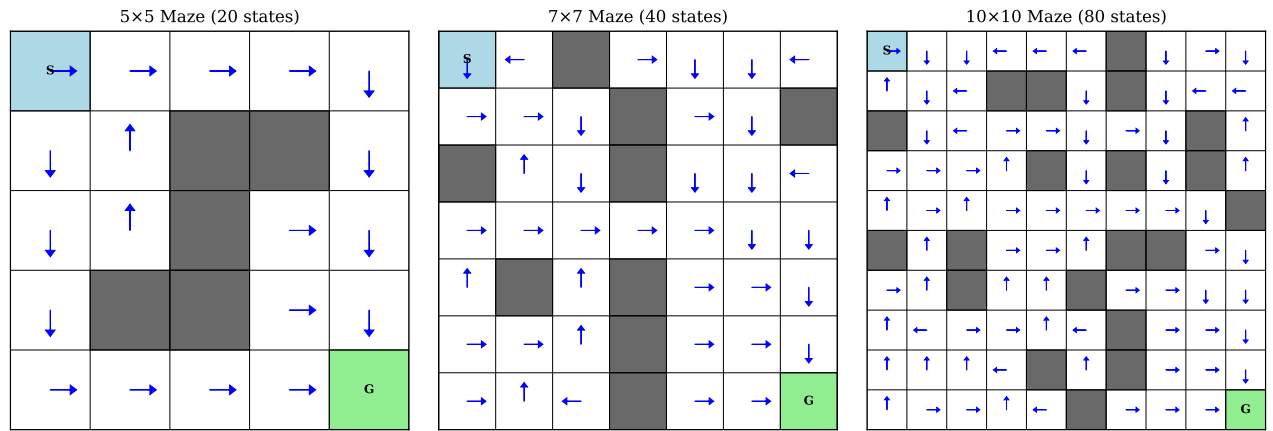


Figure 3: Learned policies for randomly generated mazes of sizes 5×5 , 7×7 , and 10×10 . The algorithm successfully learns optimal navigation policies for all maze sizes.